Rubric Writing V2

Note: This doc has undergone some changes on April 10. Please read the change log below carefully!

💡 Change Log Last Updated: Apr 10, 2026

Date	Type	Description
Apr 10, 2026	Default Tools	Added a table that explains the default and non-default tools for each universe.
Mar 25, 2026	Rubrics	**Atomic rubrics for multiple write actions of the same type:** When the prompt asks for multiple write actions (e.g., update all tickets, create tickets for all follow-up items), write one Outcome rubric per item grounded in GT — never bundle into "at least N" thresholds. "At least one/N" is reward-hackable and only acceptable when GT is genuinely indeterminate.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

Mar 12, 2026	Rubrics	**Update to Rubrics & Examples:** 1. Outcome-first: write all write actions as Outcome rubrics first; key facts reported to user go into 2.1. 2. Outcome categories simplified from 5 to 3 (1.1 write-action results, 1.2 action content, 2.1 key facts). 3. Process Rubric Decision Matrix and Default/Non-Default Tool list added to decide when to write TS and QC rubrics. 4. TS one-per-tool rule: one TS rubric per non-default tool regardless of how many times it's called. 5. All TS/QC examples updated to show only read/lookup tools — write actions never appear in TS/QC context. 6. Approximately rule updated: use for calculated/rounded values; do not use for counts, IDs, dates, or discrete quantities.
Mar 12, 2026	Failure Rate	Difficulty target clarified: 0% pass rate is acceptable as long as overall rubrics are high-quality and don't lead to false negatives/invalid model failures.

What Are Rubrics?

Rubrics are specific, checkable statements about what the AI agent should (and should not) accomplish. Each rubric is a simple yes-or-no claim. An LLM judge reads the agent's trajectory and checks each rubric against what it finds there.

You write the rubrics. The judge reads the trajectory. Pass or fail.

How the Judge Works

The judge sees:

- The original task prompt — what the agent was asked to do
- The agent's trajectory — every tool call, its parameters, its response, and the agent's reasoning at each step
- The agent's final response — the summary text at the end
- Your rubric criteria

The judge does NOT see:

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

- The MCP environment directly (it cannot call tools itself)
- The universe data (it only knows what appeared in the trajectory)
- Other agents' results

What this means for you: Every expected value must be in the rubric itself. If you write "email sent to the CEO," the judge doesn't know who the CEO is. Write "email sent to elena.rostova@moveops.com (CEO)."

Three Rubric Categories

Every rubric belongs to one of three categories: Tool Selection (TS), Query Construction (QC), and Outcome. These categories provide downstream ML teams with labeled training signals.

Core rule: Write Outcome rubrics first. Then add TS and QC only for what Outcome doesn't already cover.

Category 1: Tool Selection (TS)

Read/lookup tool calls only — never write actions.

What it checks: Did the agent use the right tool?

Write a TS rubric only when the agent might plausibly skip the tool. Not all tools have equal discriminating power — see the Default vs Non-Default Tool List below.

- Skip TS for default tools — the agent will always use these, so a TS rubric provides no signal
- Write TS for non-default tools — the agent might skip these entirely, so TS has real discriminating power

For each non-default tool, write one TS rubric regardless of how many times the agent calls it. Calling `crm_search_contacts` three times still only warrants one TS rubric.

Format:

```
The model must use the crm_search_contacts tool.

The model must use a CRM search tool (e.g., crm_search_contacts or
crm_search_companies).
```

When multiple non-default tools could achieve the same goal, use "or" to list valid alternatives.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

Default vs. Non-Default Tool Tables

- **In MoveOps tasks (v2.1 tasks or lower)**

Default (skip TS)	Non-Default (write TS as agent might skip)
search_emails	airtable_*
conversations_search_messages (Slack)	quickbooks_*
conversations_history (Slack)	crm_*
contacts_get_contacts	calendar_*
contacts_search_contacts	linear_*

- **In Keystone tasks (v2.2 tasks or above)**

Default (skip TS)	Non-Default (write TS as agent might skip)
*_email	mortgage_los_*
conversations_* (Slack)	stripe_*
contacts_*	crm_*
	filesystem_*
	quickbooks_*

Category 2: Query Construction (QC)

Read/lookup tool calls only — never write actions.

What it checks: Did the agent pass the correct parameters to critical tool calls?

Write a QC rubric only when **both** of the following are true: (1) the agent had to figure out what to query — the value wasn't handed to it in the prompt, and (2) a bad query or missing filter could cause it to miss critical data entirely. If either condition doesn't hold, skip QC.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

QC rubrics use the EM/Fuzzy pattern (see Handling Flexibility).

Format — Exact Match (structured parameter with one correct value): `The model must pass "completed" as the state filter when calling linear_list_issues.`

Format — Fuzzy (freetext parameter with multiple valid expressions): `The model must pass a full_name related to "Noah Fitzgerald" or "Fitzgerald" (or similar) when calling crm_search_contacts.`

Note on atomicity: QC rubrics may check multiple parameters of the same tool call in a single rubric. Parameters of the same tool call are tightly coupled — they describe one action.

The key distinction between QC and Outcome:

- QC checks parameters: *"The model must pass a query related to 'past 6 months' when calling linear_list_issues."*
- Outcome checks results: *"A calendar event was created (via calendar_add_calendar_event) on Feb 24, 2026 with Chloe and Elena as attendees."*

Category 3: Outcome

What it checks: What was accomplished? What does the user see?

This is where the majority of your rubrics live. Outcome rubrics describe what should have been achieved — verified from the trajectory, the final response, or both.

Target Split

Aim for 50/50 to 80/20 outcome-to-process (where TS + QC = process). Investigation-heavy tasks skew toward outcome with more 2.1 rubrics; write-action-heavy tasks have more 1.1 and 1.2 Outcome rubrics, not more QC.

The Outcome-First Workflow

Follow this sequence every time you write rubrics:

- Step 1 — Write all Outcome rubrics first: for every write action in your OEs, write 1.1 (action result) + 1.2 (content, if specific requirements exist). For every key fact the user asked to be told directly, write 2.1 (final response).
- Step 2 — From your Oracle Events, identify every non-write-action tool call. These are your candidates for process rubrics.
- Step 3 — For each candidate, apply the Process Rubric Decision Matrix to decide whether to write TS, QC, both, or neither.

If Outcome already confirms a tool was called with the right parameters, do NOT write a TS or QC rubric for that tool.

Process Rubric Decision Matrix

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

Situation	Write TS?	Write QC?	Example
Non-default tool + query value given in prompt	✅ Yes	❌ No	CRM lookup for "Noah Fitzgerald" — agent might skip CRM entirely, but the name is given in the prompt
Non-default tool + agent must infer or construct query	✅ Yes	✅ Yes	QuickBooks search for a billing dispute — agent might skip AND might search "Heartland crating" instead of "Heartland billing"
Default tool + agent must infer or construct query	❌ No	✅ Yes	Slack search for a buried stipend — agent always searches Slack, but a narrow query misses the thread
Default tool + query value given in prompt or obvious	❌ No	❌ No	Email search for Robert Calloway — agent always searches email, and "Robert" or "Calloway" both return it

---

Writing Outcome Rubrics

Outcome rubrics use three sub-categories. Use only the ones that apply to your task.

What NOT to Write

Two sub-categories from older versions of this doc are no longer required:

- Grounding checks (trajectory verification of retrieved data) — skip these. Write Outcome rubrics based on what the agent reports and does, not on verifying retrieval steps.
- Confirmation of actions in the final response — skip these. If 1.1 passes, the action happened. A separate final response confirmation adds no signal.

1.1 — Write-Action Results

Did the right action happen with the right details? Verified from the trajectory (tool call).

```
An email was sent (via send_email) from fatimah.al-rashidi@moveops.com
to chloe.vance@moveops.com.
```

Use for: every write action (linear_create_issue, crm_update_deal_stage, airtable_create_record, send_email). Always required when a write action exists.

1.2 — Action Content

Does the content match what was needed? Verified from the trajectory (tool call parameters).

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

```
The email body (visible in send_email parameters) includes the
recommended alternative city name and a cost comparison referencing
the approximately $15,000 original budget.
```

Use when: the write action has specific content requirements to be met. Only write 1.2 if it adds a distinct check beyond 1.1.

2.1 — Key Facts / Findings (Final Response)

Did the agent correctly report the right information to the user? Verified from the final response text.

```
The final response lists Noah Fitzgerald (GreenStack Solutions) as
relocating to Seattle.
```

Use when: the user asked to be told a specific fact — whether or not the task also includes write actions.

Which Sub-Categories Apply

Task Type	1.1	1.2	2.1
Pure write action (send email, create issue)	✅ Always	✅ If content requirements exist	❌
Pure investigation / summary to user	❌	❌	✅
Mixed: research then write	✅	✅	✅ Only if user also asked to be told findings directly

---

Using Tool Names in Rubrics

Context	Rule
Task prompts	Never mention tool names
TS rubrics	Always mention tool names — that's the point
QC rubrics	Mention the tool name to specify which tool call the parameters apply to
Outcome rubrics	Reference tool names with "(via tool_name)" when it adds precision

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

✅ Good: `An email was sent (via send_email) to chloe.vance@moveops.com.`
❌

Vague: `The agent sent an email to Chloe.`

Handling Flexibility: How Strict Should a Rubric Be?

Before writing each rubric, ask: does this value have exactly one correct form, or could the agent get it right a different way?

- **Strict (Exact Match)** — one correct answer. Use for: email addresses, dates, IDs, specific numbers from tool outputs, tools with no alternatives.
- **Flexible** — multiple valid expressions. Use for: freetext search queries, email subjects, issue titles, tools with alternatives, agent-generated content.

Applying Strict vs. Flexible to Each Category

TS rubrics — read/lookup tools only, never write actions:

Strict — only one tool works: `The model must use the crm_search_contacts tool.`

Flexible — multiple non-default tools could each achieve the goal: `The model must use a CRM search tool (e.g., crm_search_contacts or crm_search_companies).`

QC rubrics — read/lookup tools only, never write actions:

Strict — structured parameter with exactly one correct value: `The model must pass "completed" as the state filter when calling linear_list_issues.`

Flexible — freetext parameter open to interpretation: `The model must pass a full_name related to "Noah Fitzgerald" or "Fitzgerald" (or similar) when calling crm_search_contacts.`

Common mistake: being too rigid on freetext. If you write "must pass 'Noah Fitzgerald'" and the agent passes "Fitzgerald," that's the same intent — use "(or similar)."

Outcome (1.1 / 1.2) — verified from trajectory:

Write-action result — Strict: `An email was sent (via send_email) to chloe.vance@moveops.com.`

Email subject — Flexible: `The email subject relates to a relocation proposal (e.g., "Relocation Proposal for GreenStack" or similar).`

Action content — Strict (exact string from data): `The email body includes the client name "GreenStack Solutions."`

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

Action content — Flexible (calculated or rounded value): `The email body includes the approximately $15,000 original budget figure.`

Action content — Flexible (Required Elements): `The email body must include: (a) the reason for the change (e.g., storm or similar), (b) the alternative city name, and (c) a cost comparison.`

Outcome (2.1) — verified from final response:

Key facts — Strict: `The final response lists Noah Fitzgerald (GreenStack Solutions) as relocating to Seattle.`

Key facts — Flexible: `The final response identifies the most cost-effective option (e.g., "Denver is the cheapest" or similar comparison).`

Quick Reference

Situation	Pattern	Example
One correct tool / value / fact	Strict (EM)	"crm_search_contacts", "chloe.vance@moveops.com"
Multiple valid non-default tools	Flexible	"crm_search_contacts or crm_search_companies"
Freetext parameter or agent-generated label	Flexible (Fuzzy)	"related to relocation proposal (or similar)"
Agent content with specific requirements	Required Elements	"must: (a) reason for change, (b) city name, (c) cost comparison"
Similar entities, one correct	Selection Logic	"priya.venkatesh@... — the Priya with a post-move thread"

Three Fields Per Rubric

Every rubric needs three fields.

1. Criterion The specific yes/no claim the judge evaluates. Must be self-contained, objective, atomic, and verifiable.

2. Justification 1–2 sentences explaining why this rubric exists. Connects to the prompt or a known failure mode. *"This rubric exists because the prompt explicitly says 'CC Elena.' If Elena doesn't see the proposal, she can't approve it."*

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

3. Evidence What to look for in the trajectory. Points to the specific tool call, parameter, or response text that proves pass or fail. *"Look for send_email tool call. Check the CC field for elena.rostova@moveops.com."*

Service Metadata Requirements

Rubrics missing metadata are incomplete. Always include the required fields per service.

Email — must include: Recipient (full email address), CC (if specified), Content (specific items the email must contain — list individually)

Slack — must include: Recipient (channel name or DM recipient), Content (specific items the message must mention — list individually)

Linear — must include: Title (what the issue title should contain), Assignee (if specified), Priority (if specified), Subtasks (count and coverage if applicable)

Incomplete	Complete
"Agent messaged David on Slack"	"Slack message to David Chen mentioning: (a) Apex AI retreat, (b) the new city, (c) the storm reason"
"Email sent about rebooking"	"Email (via send_email) to chloe.vance@moveops.com, CC elena.rostova@moveops.com, containing alternative city name and budget comparison to approximately $15,000"

How to Write Good Rubrics — Rules

Rule 1: Be Self-Contained

The judge can't look anything up. Include all expected values.

Bad	Good
"Email sent to the CEO"	"Email sent to elena.rostova@moveops.com (CEO)"
"Budget is correct"	"Total cost is approximately $15,000 or less"
"Slack message has enough context"	"Slack message mentions: (a) Apex AI, (b) new city, (c) storm"

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

"Agent contacted the right person"	"Email sent to chloe.vance@moveops.com"

Rule 2: One Thing Per Rubric, No Overlaps, No Gaps

Each rubric must check exactly one independent claim.

Bad	Good
"Email mentions storm, new city, and flight details"	Three rubrics: (1) email mentions the storm / (2) email includes the new city / (3) email includes flight details
"Linear issue created for Fatimah and email sent to Chloe"	Two rubrics: (1) Linear issue created for Fatimah / (2) email sent to Chloe

Bundling exception: Tightly coupled facts may be bundled in two cases — (a) same tool call: parameters checked together on one call (e.g., "An email was sent (via send_email) to chloe.vance@moveops.com, CC elena.rostova@moveops.com" — both are visible in the same send_email parameters); or (b) same data point: facts from a single record that are meaningfully inseparable (e.g., "The final response lists Noah Fitzgerald (GreenStack Solutions) relocating to Seattle" — name, company, and city all come from the same CRM record and would pass or fail together). Use judgment: if two claims could plausibly fail independently, split them.

Rule 3: Must Be Verifiable from the Trajectory or Final Response

Bad (unverifiable)	Good (verifiable)
"Email exists in SENT folder"	"An email was sent (via send_email) to chloe.vance@moveops.com"
"Calendar event was created in Google Calendar"	"A calendar event was created (via calendar_add_calendar_event) on Feb 24, 2026 with Chloe and Elena as attendees"
"The agent understood the problem"	"The final response correctly identifies the root cause as the duplicate flight confirmation"

Rule 4: Use "Approximately" for Calculated or Rounded Numbers

✅ `Cost is approximately $12,500` ✅ `Cost is between $12,000 and $13,000`
❌ `Cost is $12,487.50` — too precise, agent won't match this exactly

Do NOT use "approximately" for fixed, static values:

- Counts: "3 overdue tasks" — not approximately 3

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

- IDs: "issue OPS-312" — not approximately OPS-312
- Dates: "February 24, 2026" — not approximately Feb 24
- Discrete quantities from the data: "5 relocations" — if the data has exactly 5, say 5

Rule 5: Handle Multiple Valid Answers Explicitly

Phrasing	Meaning
"must be one of: Austin, Denver, or LA"	Closed — only these are correct
"including but not limited to: Austin, Denver"	Open — others also acceptable
"at least one of: Austin, Denver, or LA"	Any single one is sufficient

Never use "such as," "like," or "for example" when defining what counts as correct — these are ambiguous. (Fine in Fuzzy rubrics where you're illustrating intent, not defining the answer set.)

Before Writing Rubrics: Plan Your Oracle Events

Before writing any rubrics, map out the Oracle Events (OEs) — the key steps a perfect agent would take to solve the task. Write these as free-form text describing:

- What critical actions need to happen (send email, create issue, etc.)
- What information needs to be discovered (which clients, what dates, what conflicts)
- What tools and parameters are needed for each key step
- What the correct final answer looks like

Oracle Events serve two purposes:

- **Prove solvability** — demonstrate that the task can be solved and that you understand the correct solution path
- **Drive the two-filter workflow** — OEs identify which actions exist (Filter 1), and the matrix decides which process rubrics to write (Filter 2)

Example OEs for *"send an email to Chloe from Fatimah about a relocation proposal"*:

OE 1: Look up Fatimah's and Chloe's contact details using contacts_search_contacts or contacts_get_contacts.

> *→ Both are default tools → skip TS. Names are given in the prompt → skip QC.*

OE 2: Send email from fatimah.al-rashidi@moveops.com to chloe.vance@moveops.com with subject and body relating to a relocation proposal.

→ Write action → goes into Outcome. No TS or QC for send_email — Outcome already covers it.

→ Outcome 1.1: "An email was sent (via send_email) from fatimah.al-rashidi@moveops.com to chloe.vance@moveops.com."

→ Outcome 1.2: "The email subject relates to a relocation proposal (e.g., 'Relocation Proposal for GreenStack' or similar)."

Writing Rubrics: Step by Step

Step 1: Identify What the Prompt Asks For

Read the prompt and list every explicit and implicit ask.

Explicit asks — directly requested:

- What actions should be taken? (send email, create issue, schedule meeting)
- What information should be reported? (findings, summaries, recommendations)
- What constraints exist? (budget limits, specific recipients, exclusions)

Implicit asks — reasonably expected but not stated:

- Units or context for numbers (dollars, dates)
- Disambiguation when multiple entities could match
- Flagging problems discovered during investigation

Step 2: Write Oracle Events

Map the key steps a perfect agent would take. For each OE, note whether it is a write action or a read/lookup action — this determines which rubric type applies.

Step 3: Write All Outcome Rubrics

For every write action in your OEs: write 1.1 (action details) and 1.2 (content, if specific requirements exist).

For every key fact the user asked to be told directly: write 2.1 rubrics.

For mixed tasks (research then write): write 1.1 + 1.2 for the write action. Add 2.1 only if the prompt also explicitly asks the agent to report findings to the user directly.

Step 4: Write Process Rubrics for Remaining Read/Lookup OEs

From your OEs, identify every non-write-action tool call. For each one, apply the Process Rubric Decision Matrix to decide: TS only, QC only, both, or neither.

Step 5: Check Flexibility

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

For each rubric, determine if the expected value is EM, Fuzzy, Selection Logic, or Required Elements. Apply the correct pattern.

Step 6: Verify Checklist

Before submitting, confirm:

- Every rubric has all 3 fields: criterion + justification + evidence
- Every rubric belongs to one of three categories: TS, QC, or Outcome
- Outcome-to-process split is between 50/50 and 80/20
- Outcome rubrics written first — process rubrics only added for what Outcome doesn't cover
- No TS or QC rubric written for a tool already covered by Outcome
- TS written only for non-default tools the agent might plausibly skip
- QC written only when agent had to figure out the query AND bad query = missing data
- Every write action has a 1.1 rubric; 1.2 if specific content requirements exist
- 2.1 used only when the user asked to be told a specific fact
- Every criterion is self-contained (all expected values embedded)
- Every criterion is objective (no banned words)
- Every criterion is atomic (one claim per rubric)
- No two rubrics penalize the same error (no overlaps)
- Every explicit prompt ask is covered by at least one rubric (no gaps)
- Implicit asks covered where they test reasoning or synthesis
- Every criterion is verifiable from the trajectory or final response
- Email rubrics include recipient, CC, and content specifics
- Slack rubrics list specific content items
- Linear rubrics include title and relevant fields
- Calculated/rounded numbers use "approximately" or a range; counts, IDs, and dates use exact values
- Fuzzy values include examples + "(or similar)"
- Multiple valid answers use "must be one of" / "including but not limited to"

Edge Cases

When the prompt specifies an explicit count or names specific items: Write one atomic Outcome rubric per item, each naming the specific ticket ID, record, or entity and what the update should reflect. Do not bundle into "at least N" thresholds.

✅ Correct - one rubric per ticket:

- "A comment was added (via `linear_create_comment`) to ENG-211 reflecting the current status of the weather API fix."
- "A comment was added (via `linear_create_comment`) to ENG-212 reflecting the current status of the duplicate receipt issue."

❌ Wrong:

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

- "At least 5 of the 9 Linear tickets were updated with current status." → Reward-hackable. Agent updates 5 arbitrary tickets and passes.

When the prompt is open-ended ("create tickets for anything needing follow-up"): Go to the universe, identify the actual GT items that need follow-up, and write one rubric per item naming it explicitly. "At least one" is only acceptable if the GT is genuinely indeterminate — which is rare. If the universe shows 3 specific items needing tickets, write 3 rubrics.

The key principle: Rubrics should be grounded in GT, not in what feels like a "reasonable threshold." An agent that updates 8 of 9 tickets should fail — the prompt asked for all 9. Gradation comes from having 9 separate rubrics, not from setting an arbitrary pass threshold on one bundled rubric.

Common Mistakes

Mistake 1: Subjective Language "Enough context," "professional tone," "thorough investigation" — replace with specific, countable items. This is the #1 mistake.

Mistake 2: Missing Service Metadata Email without recipient. Calendar without attendees. Slack without specific content items.

Mistake 3: Can't Verify from Trajectory "Email exists in the SENT folder" — the judge reads the trajectory, not the environment. See Rule 3.

Mistake 4: All Rubrics Check the Same Thing If all your rubrics are "final response mentions X," you're missing write-action coverage. Add 1.1 and 1.2 rubrics for any write actions in the task. Cover multiple dimensions.

Mistake 5: No Justification Every rubric needs a WHY. Helps reviewers understand your reasoning and catch gaps.

Mistake 6: Stacked Rubrics "Email sent to Chloe and CC'd to Elena with storm explanation and budget comparison" — 4 rubrics in 1. Split independent claims. Exception: tightly coupled facts from the same tool call or data point can be bundled.

Mistake 7: Writing TS/QC for Write Actions If Outcome already confirms a tool was called with the right parameters, do not write TS or QC for that tool. Outcome first — process only for gaps.

Mistake 8: TS for Default Tools Writing TS for `search_emails`, `conversations_search_messages`, `conversations_history`, `contacts_get_contacts`, or `contacts_search_contacts` provides no signal — the agent always uses these. Save TS for non-default tools (`airtable_*`, `quickbooks_*`, `crm_*`, `calendar_*`, `linear_*`). See the Default vs Non-Default Tool List.

Mistake 9: Ordering Rubrics in Disguise "The agent must use crm_search_contacts before search_emails" — this checks execution order. TS checks tool selection. QC checks parameters. Neither checks ordering.

Mistake 10: Ambiguous Tool References "The agent sent an email" — too vague. Write "An email was sent (via send_email)" when the mechanism matters.

Mistake 11: Overlapping Rubrics If the agent gets the recipient wrong, how many rubrics fail? If more than one, you have overlaps. Each error should be caught by exactly one rubric.

Difficulty Target

We run the agent 6 times with Opus 4.6. Your task is ready when:

- All fails is good (pass@1 = 0%)
- At most 2 runs pass — pass@1 ≤ 40% (proves difficulty)

All 6 pass → too easy, iterate. All 6 fail is fine (pass is 0%).

Tip: Use Haiku for quick iteration. Switch to Opus 4.6 for the final 6 runs.

The examples below show how to write rubrics for simple prompts. These aren't exhaustive and don't ensure the accuracy/high-quality we expect from your tasks.

Example: Simple Task

Prompt: *"Send an email to Chloe from Fatimah about a relocation proposal"*

Outcome

#	Criterion
1	An email was sent (via send_email) from fatimah.al-rashidi@moveops.com to chloe.vance@moveops.com. [1.1 — write-action result]
2	The email subject (visible in send_email parameters) relates to a relocation proposal (e.g., "Relocation Proposal for GreenStack" or similar). [1.2 — action content]

Distribution: TS: 0, QC: 0, Outcome: 2 — process 0%, outcome 100%

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

No TS section: contacts_search_contacts and contacts_get_contacts are both default tools — no TS rubric. No QC section: names are given in the prompt → skip QC. send_email is a write action → covered by Outcome. No 2.1: the deliverable is the email, not a summary to the user.

Example: Task 2

Prompt: *"Search for all of Fatimah's relocations from the past 6 months and compile a summary" (Universe data: Fatimah has 5 relocations — Fitzgerald/Seattle, Fitzpatrick/Austin, Venkatesh/Portland, Webb/Denver, Voss/Chicago)*

Tool Selection

#	Criterion
1	The model must use the crm_search_contacts tool (or equivalent CRM lookup). [Non-default — agent might skip entirely. "Fatimah" is given in the prompt → TS only, skip QC]

Outcome

#	Criterion
2	The final response lists Noah Fitzgerald (GreenStack Solutions) relocating to Seattle. [2.1]
3	The final response lists Noah Fitzpatrick (Axiom Labs) relocating to Austin. [2.1]
4	The final response lists Priya Venkatesh (GreenStack Energy) relocating to Portland. [2.1]
5	The final response lists Marcus Webb (Helix Corp) relocating to Denver. [2.1]
6	The final response lists Elena Voss (Pinnacle) relocating to Chicago. [2.1]
7	The final response identifies the approximate time range covered (roughly September 2025 through present). [2.1]
8	The final response includes the status of each relocation (e.g., completed, in progress, or pending — specific statuses should match universe data). [2.1]

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z

9	The final response does not include relocations from more than 6 months ago. [2.1 — negative rubric for date constraint]

Distribution: TS: 1, QC: 0, Outcome: 8 — process 11%, outcome 89%

No QC: "Fatimah" is given in the prompt — agent can't meaningfully get the query wrong. No CRM tool supports date filtering as a parameter — the date constraint is enforced via Outcome rubric 9, not QC. Pure investigation — no write action, so no 1.1 or 1.2. Final response is the deliverable, so 2.1 carries everything.

When to Write Rubrics

Write rubrics during and after the agent run, not before.

1. Write your prompt and submit it
2. The agent runs
3. While the agent runs, start drafting rubrics based on what you expect should happen
4. After the agent finishes, review the trajectory
5. Finalize rubrics — adjust based on what you actually see
6. The LLM judge grades the rubrics against the trajectory

You can iterate: tweak prompt → re-run → adjust rubrics. If the agent passes everything, the task isn't hard enough — iterate.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.973Z